FUNDAMENTAL DEVELOPMENT

Software Development

© TestHuset A/S - 2026

HI! I'M ALEX STORM

Consultant • Teacher • Software Engineer

EXPERIENCE
10+ years building and testing systems for Danish companies

ASK ME ABOUT
C#, Python, Typescript, Test Automation, Clean code, AI

FUELED BY
Coffee (self-proclaimed enthusiast)



ast@testhuset.dk • linkedin.com/in/stormeal

GENERAL INFORMATION



AGENDA

01 **WHAT IS SOFTWARE DEVELOPMENT**
We'll dive deep into what software development is, and some of the myths around the topic.

02 **METHODOLOGIES**
Various methodologies surrounding software development.

03 **PROGRAMMING FOUNDATION**
We'll dive into the topic programming and what coding is.

04 **INTRODUCING PYTHON**
Why are we going to use Python in this course, some of the benefits and disadvantages

05 **BASIC SYNTAX**
We must start somewhere; this is one of the first steps when learning a programming language

06 **CONDITIONS**
We teach our programs to make decisions and react differently based on the situation

07 **LOOPS**
We make our programs repeat work automatically, instead of writing the same thing over and over.

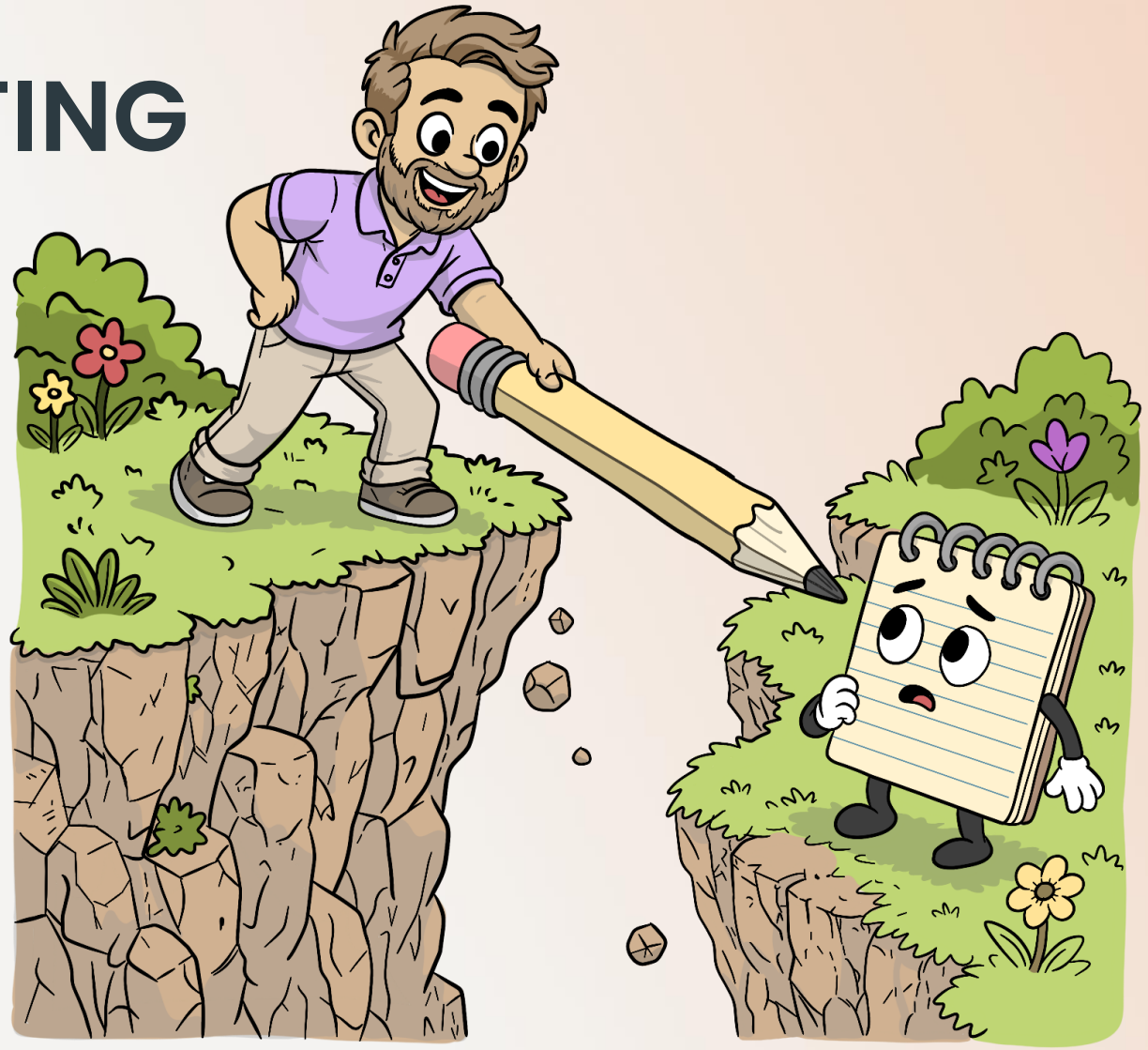
08 **FUNCTIONS**
We package our code into reusable named blocks, keeping it tidy and easy to change.

THE GROUND IS SHIFTING

- Companies are upskilling their testing teams fast. Technical skill is no longer a nice-to-have, it is becoming the baseline.
- Many skilled technical testers never learned to program. As automation and AI take over more of the work, that gap is starting to show.
- Developers hand over setups, tools, and ideas that assume you can read code. Without that foundation, it is easy to fall behind, even for experienced testers.
- This course closes that gap. You learn how software is built from the inside, so you can keep up, contribute more, and work as an equal alongside developers.

THE GROUND IS SHIFTING

- 58% of companies are upskilling their QA teams in AI and new tech *(World Quality Report 2025)*
- ~80% of the workforce will need upskilling by 2027 due to AI *(Gartner)*
- The industry is shifting from classic QA toward more technical testing roles (SDET)



WHAT IS SOFTWARE DEVELOPMENT

— Common myths and activities performed by a software developer


CARS
PHONES
COMPUTERS
SMART LIGHTS
SOFTWARE IS EVERYWHERE
22°
SMART THERMOSTATS
120
WEARABLES
FRIDGES
TOASTERS
COFFEE MACHINES
SMART TVS
WASHING MACHINES

SOFTWARE DEVELOPER

- **Core Responsibilities**
  - Coding and troubleshooting
  - Design & Architecture
  - Testing
  - Maintenance









In other words
Person responsibile for creating, designing, testing and also maintaining the actual software



- ✗ PROGRAMMERS WORK ALONE
- ✗ YOU NEED A DEGREE
- ✗ ONLY FOR INTROVERTS
- ✗ IT'S TOO LATE TO START



- ✓ TEAMWORK & COLLABORATION
- ✓ PROBLEM SOLVING & CREATIVITY
- ✓ DIVERSE SKILLS & PERSPECTIVES
- ✓ BUILDING IMPACTFUL SOLUTIONS

WHAT ACTUALLY MATTERS?

- Curiosity and willingness to learn
- Problem-solving skills
- Communication and teamwork
- Persistence when facing challenges
- Ability to adapt to new technologies
- Understanding user needs



In other words
Coding is important, but these skills often matter even more.

SKILLS TRANSFER ACROSS CAREERS

- **Teacher** -> Explaining complex ideas clearly
- **Chef** -> Following processes and multitasking
- **Mechanic** -> Troubleshooting complex systems
- **Manual Tester** -> Debugging and quality assurance



Software development isn't a profession reserved for people who have been coding since they were teenagers. Many successful developers started in completely different careers. The skills they developed was communication, troubleshooting, organization, creativity, and quality-focused thinking often become some of their greatest strengths.

YOU ARE DONE LEARNING

- In software development you are never truly done learning. It's an adaptive field that changes every day.
- Keeping your knowledge up to date is imperative
  - Learning new programming languages
  - Explore new frameworks
  - Solve programming challenges

SOFTWARE DEVELOPMENT IS BORING?

- Doing the same task every day can tend to be boring. But software developers wear many different hats, and it's not only coding 8hrs every day.

- **Various tasks**
  - Problem solver
  - Analyst
  - Designer
  - Tester
  - Coach
  - Tech reviewer

ACTIVITIES IN SOFTWARE DEVELOPMENT

- **Collecting and analyzing requirements**
- Validating ideas, performing feasibility studies
- Defining the software / project architecture
- Implementation and Quality Assurance
- Writing documentation and guides
- Releasing, supporting and maintaining the software



COLLECTING AND ANALYZING REQUIREMENTS

- Understand ***what*** the software must do, before deciding ***how***
- Gather needs from stakeholders, users, and the business
- Turn vague wishes into clear, testable requirements
- Disinguish functional (“the system shall..”) from non-functional (speed, security, usability)



ACTIVITIES IN SOFTWARE DEVELOPMENT

- Collecting and analyzing requirements
- **Validating ideas, performing feasibility studies**
- Defining the software / project architecture
- Implementation and Quality Assurance
- Writing documentation and guides
- Releasing, supporting and maintaining the software



VALIDATING IDEAS

- Ask: ***should*** we build this, and can we?
- Check feasibility across technical, financial and time dimensions
- Validate the ideas with users early (prototype, mockups)
- Cheaper to discard a bad idea now than after months of coding



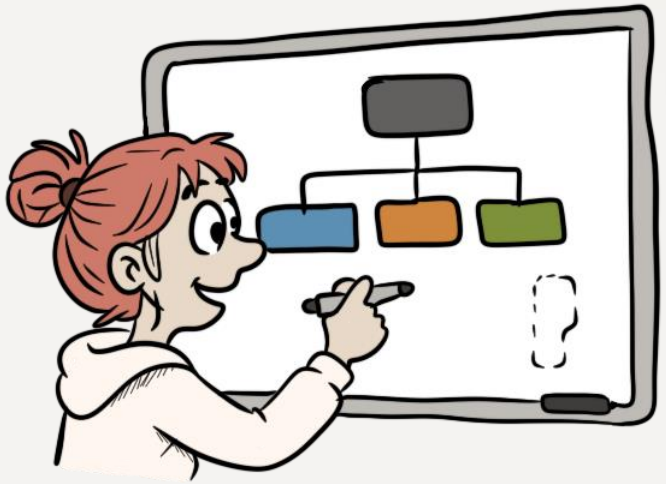
ACTIVITIES IN SOFTWARE DEVELOPMENT

- Collecting and analyzing requirements
- Validating ideas, performing feasibility studies
- **Defining the software / project architecture**
- Implementation and Quality Assurance
- Writing documentation and guides
- Releasing, supporting and maintaining the software



DEFINING THE ARCHITECTURE

- The blueprint: how the system is structured and how parts connect
- Choose technologies, frameworks, and patterns
- Plan for scalability, maintainability, and security from the start
- Good architecture makes future change cheap; poor architecture makes it painful



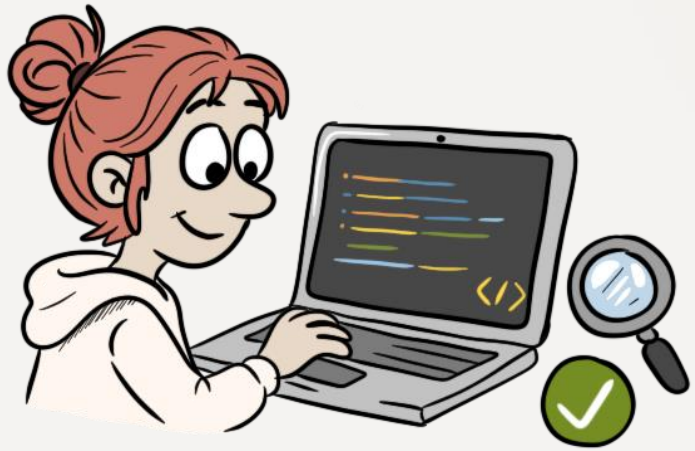
ACTIVITIES IN SOFTWARE DEVELOPMENT

- Collecting and analyzing requirements
- Validating ideas, performing feasibility studies
- Defining the software / project architecture
- **Implementation and Quality Assurance**
- Writing documentation and guides
- Releasing, supporting and maintaining the software



IMPLEMENTATION AND QUALITY ASSURANCE

- Implementation means writing the actual code
- Follow standards, use version control, review each other's work
- QA runs in parallel. Testing is not an afterthought
- Catch defects early with unit tests, integration tests, and manual testing.



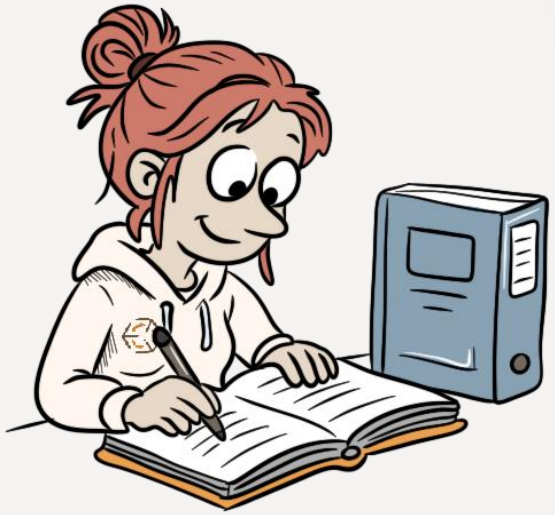
ACTIVITIES IN SOFTWARE DEVELOPMENT

- Collecting and analyzing requirements
- Validating ideas, performing feasibility studies
- Defining the software / project architecture
- Implementation and Quality Assurance
- **Writing documentation and guides**
- Releasing, supporting and maintaining the software



WRITING DOCUMENTATION AND GUIDES

- Documentation makes software usable and maintainable
- Two audiences: **users** (how to use it) and **developers** (how it works)
- Includes API documentation, setup guides, user manuals and inline comments
- “Future you” and new team members will thank you



ACTIVITIES IN SOFTWARE DEVELOPMENT

- Collecting and analyzing requirements
- Validating ideas, performing feasibility studies
- Defining the software / project architecture
- Implementation and Quality Assurance
- Writing documentation and guides
- **Releasing, supporting and maintaining the software**



RELEASING, SUPPORTING & MAINTAINING

- Release: deliver the software to its users
- Support: help users, fix issues, respond to feedback
- Maintenance: bug fixes, updates, and new features over time
- Software is never truly “finished” it lives and evolves



RECAP: SOFTWARE DEVELOPMENT

- A developer designs, builds, tests and maintains software, not just writes code
- The myths are wrong: it is teamwork, and no degree or personality type is required
- Curiosity, problem solving and communication matter as much as coding
- Skills from other careers transfer, and you are never done learning
- Six activities: requirements, validation, architecture, implementation and QA, documentation, release and maintenance



Key takeaway
Software development is a team effort to solve problems. Coding is only one part of it.

SOFTWARE DEVELOPMENT METHODOLOGIES

- **What is the methodologies?**
  - Waterfall Model
  - Agile Software Development

WATERFALL

- A linear, sequential approach: each phase finishes before the next begins
- Classic phases: Requirements, Design, Implementation, Testing, Deployment, Maintenance
- Like water flowing downhill, you move in one direction and rarely go back
- Heavy emphasis on planning and documentation up front



WATERFALL

- ✓ Simple to understand and manage; clear structure.
- ✓ Well-defined stages, milestones and deliverables
- ✓ Strong documentation, easy to onboard new people
- ✓ Works well for fixed, well-understood requirements



- X Inflexible; hard to react to changing requirements
- X Working software appears only late in the project
- X Bugs and wrong assumptions surface late, when they cost the most.
- X Poor fit for large, long or uncertain projects

AGILE

- An iterative approach: build in small cycles called iterations or sprints
- Deliver working software frequently, then gather feedback and adjust
- Values collaboration, customer involvement, and responding to change
- Grounded in the Agile Manifesto; common frameworks include Scrum and Kanban



AGILE

- ✓ Flexible; welcomes changing requirements, even late
- ✓ Delivers working software early and continuously
- ✓ Close customer collaboration keeps the product on target
- ✓ Problems surface quickly while they are still cheap to fix



- X Less predictable on final cost, scope, and timeline
- X Needs strong communication and an engaged customer
- X Lighter documentation can be a risk if overdone
- X Can drift without discipline and experienced people

WHICH ONE SHOULD YOU USE?

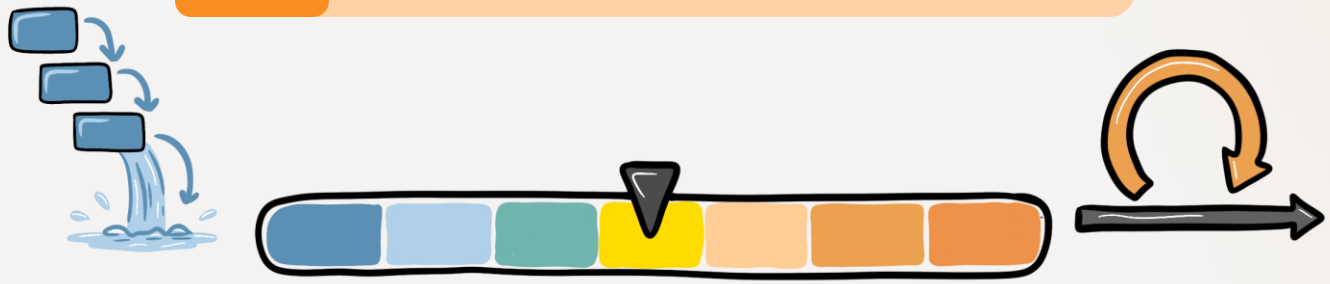
Lean waterfall when:

- Requirements are clear, stable and unlikely to change
- Scope and budget are fixed up front
- The project is short and well understood

Lean Agile when:

- Requirements are unclear or expected to evolve
- The customers wants to be involved and see progress early
- The project is large, long or exploratory

Many real teams blend both, using structure where it helps and flexibility where it counts. No team or project is the same.



RECAP: METHODOLOGIES

- A methodology is the way a team organises the work of building software
- Waterfall is linear: each phase finishes before the next one begins
- Waterfall suits clear, stable requirements with fixed scope and budget
- Agile is iterative: small sprints, working software early, frequent feedback
- Agile suits unclear or changing requirements and involved customers



Key takeaway
Neither is better by default. Pick what fits the project; many teams blend both.

PROGRAMMING FOUNDATION

- What is Coding
- Programming Languages

PROGRAMMING: What is coding?

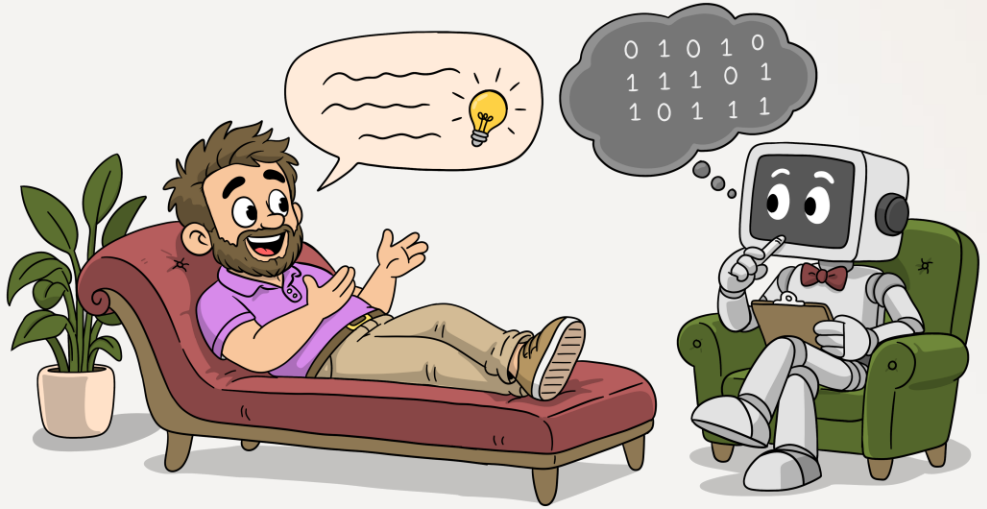
- Instruct a computer to execute a specific tasks.
- Every program is a sequence of commands (instructions) the computer runs in order.
- You write those instructions in a programming language, a set of words and rules the computer can follow.
- The computer does *exactly* what you say; no guessing, no “you know what I mean”
- Get one instruction wrong and the program does the wrong thing, or stops. That is normal, and fixing it is part of the job.

THE COMPUTER IS NOT SMART, JUST FAST

- A computer has no common sense. It does only what it is told, in the exact order it is told.
- Tell a person “Get me a drink” and they fill in the gaps. A computer needs every step spelled out.
- One missing or wrong step and it stops, or it does the wrong thing perfectly.

HOW WE TALK TO A COMPUTER

- Humans speak English, Norwegian, gestures. Computers ultimately understand only 1s and 0s (machine code).
- A programming language sits in the middle. It is readable enough for humans to write and strict enough for machines to run.
- Examples: Python, JavaScript, C#. In this course we use Python.



YOUR FIRST INSTRUCTION

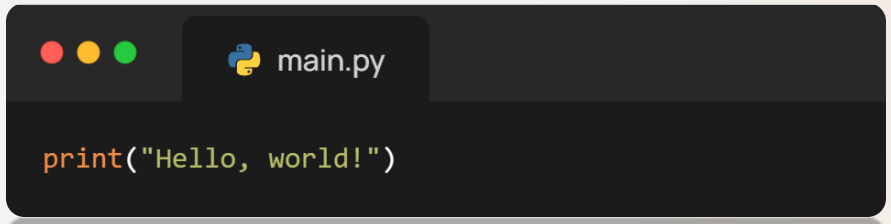
- Here is a complete, working program



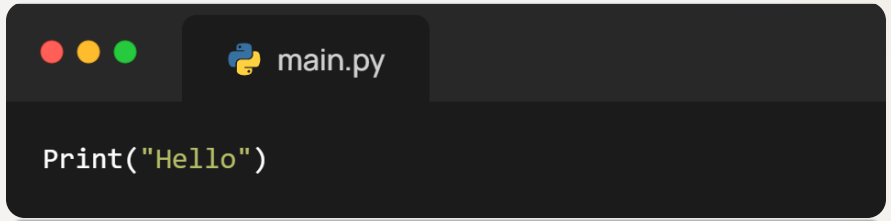
- print is a command. It means "show this text on the screen."
- That is it. The computer runs the line, shows the text, and is done.

EXACT MEANS EXACT

— This works



— This does not

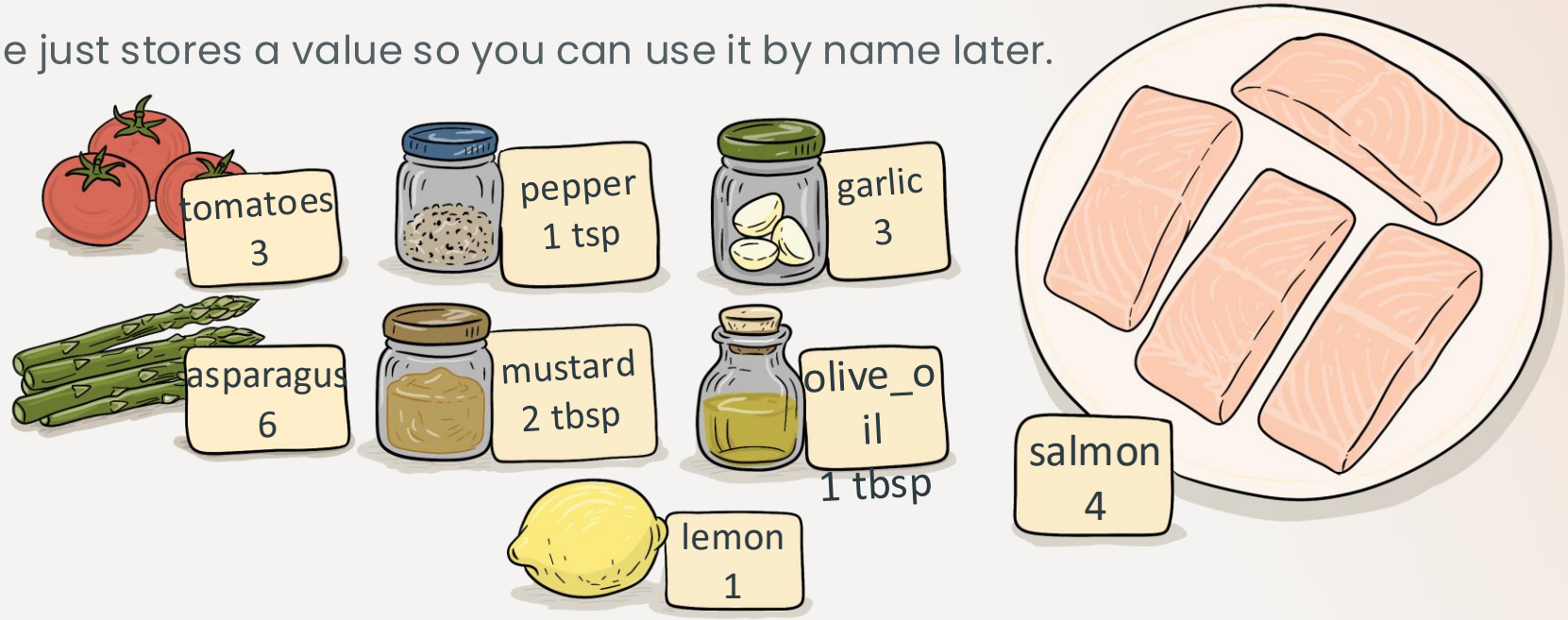


— The only difference is a capital P. The computer will not guess what you meant. It reports an error and stops.

A PROGRAM IS LIKE A RECIPE

Ingredients are variables

- In a recipe, each ingredient has a name and an amount. In code, that is a variable (the name) and its value (the amount).
- The label tells you what it is. The value tells you how much.
- A variable just stores a value so you can use it by name later.



TestHuset

A PROGRAM IS LIKE A RECIPE

Steps are the instructions

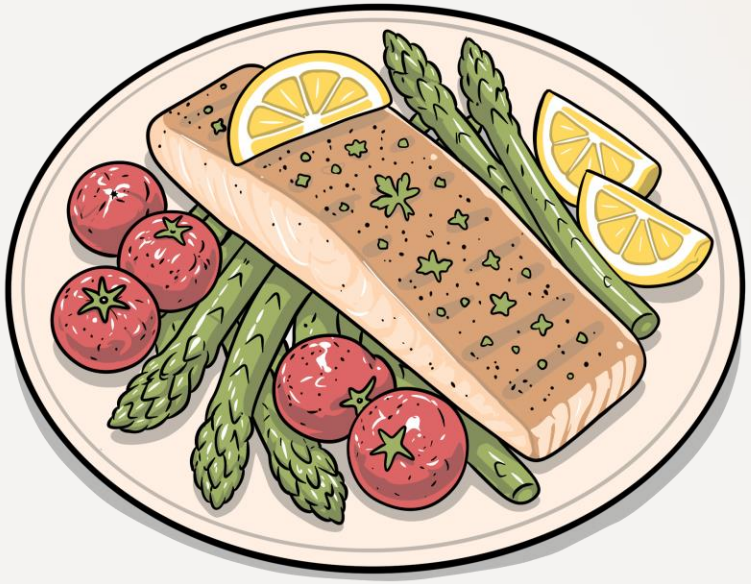
- A recipe is a list of steps, done in order. A program is a list of instructions, run in order.
- Each numbered step is one instruction. The computer does step 1, then step 2, then step 3, and so on.
- The steps use the ingredients you defined. The code uses the variables you defined.



A PROGRAM IS LIKE A RECIPE

The dish is the output

- Follow the recipe and you get a finished dish. Run the program and you get an output.
- Same ingredients and same steps give the same result every time. That is what "the computer does exactly what you say" means in practice.
- Change an ingredient or a step, and the output changes.



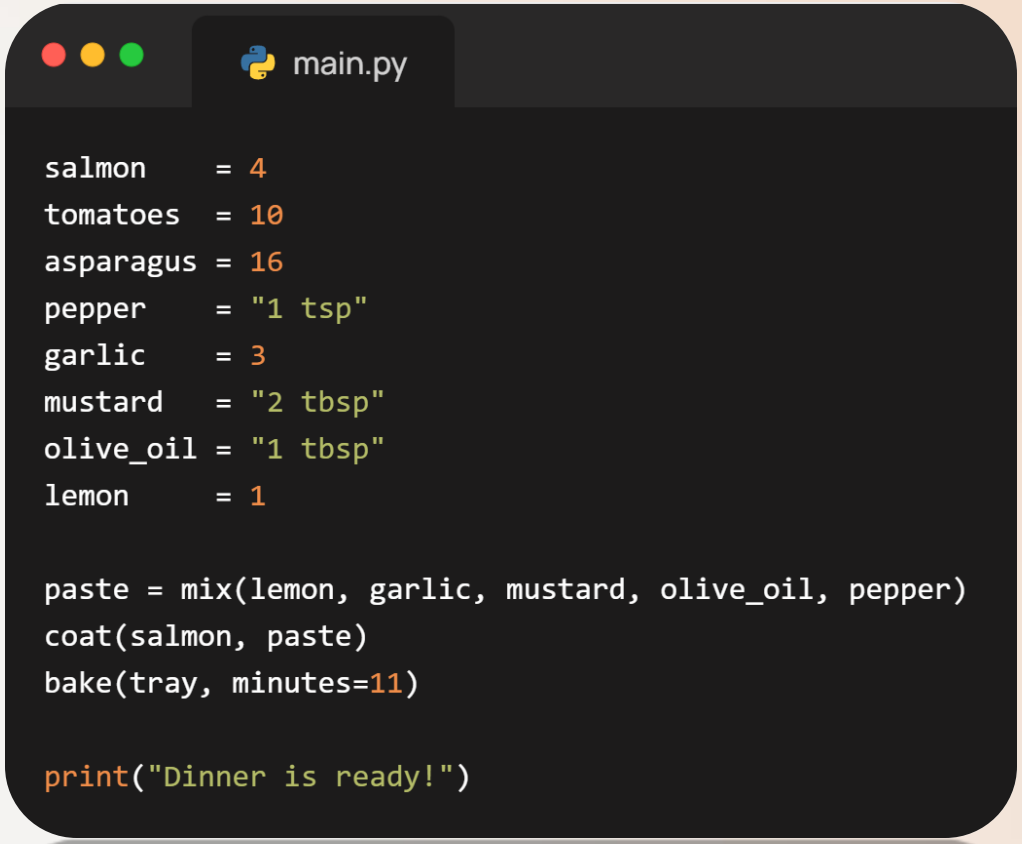
A PROGRAM IS LIKE A RECIPE

The same idea, written as real code

Everything you just saw as a recipe is real programming.

Here it is side by side:

- **Recipe (what we did)**
  - salmon, 4 fillets
  - tomatoes, 10
  - asparagus, 16
  - pepper, 1 tsp
  - lemon, 1…
  - Mix the paste, coat the salmon, bake it
  - Out comes the dish



Ingredients become variables. Steps become instructions. The finished dish becomes the output

TestHuset

PROGRAMMING: Programming Languages?

- A programming language is a collection of keywords and rules for instructing a computer.
- The keywords are the words the language understands (like if, while, print).
- The rules (called syntax) decide how those words must be put together. Break a rule and the program will not run.
- Different languages exist for different jobs, but they all share the same purpose: tell the computer what to do in a way it can be translated to follow.

SAME TASK, LANGUAGE EXAMPLES

Convert a temperature from Fahrenheit to Celsius:

Example C#:

C# temperature_controller.cs

```csharp
double fahrenheit = 98.6;
double celsius = (fahrenheit - 32) * 5 / 9;

Console.WriteLine(celsius);
// Output: 37
```

Example Python:

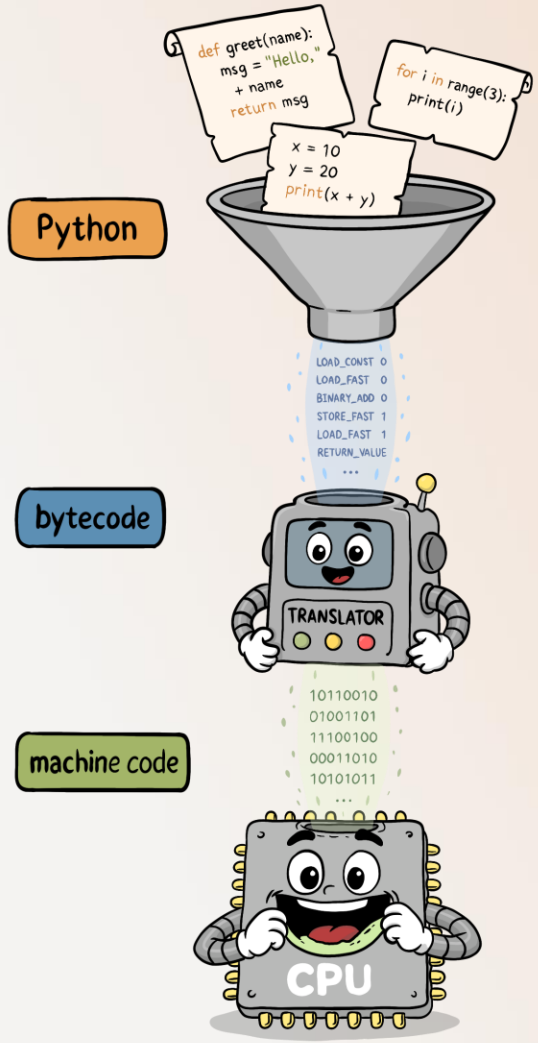
main.py

```python
fahrenheit = 98.6
celsius = (fahrenheit - 32) * 5 / 9

print(celsius)
# Output: 37.0
```

DOES THE CPU UNDERSTAND PYTHON OR C#?

- No. The CPU does not understand Python, C#, or any high-level language.
- A CPU can only execute machine code: raw instructions made of numbers, specific to that exact type of hardware.
- Machine code is not one universal thing. An Intel chip and a phone's ARM chip speak different machine code.
- So how does our Python or C# ever run?



TestHuset

WHY WE USE HIGH-LEVEL LANGUAGES

- A translator program does the hard part for us. A compiler or an interpreter turns our code into the right machine code for the hardware it runs on.
- We write one readable version. The translator handles the messy, hardware-specific details.
- This means we do not have to rewrite our program for every different CPU. The same Python or C# can run on a laptop, a server, or a phone.
- High-level means "close to how humans think." Low-level means "close to the hardware." We work high so we can focus on the problem, not the chip.

RECAP: PROGRAMMING FOUNDATION

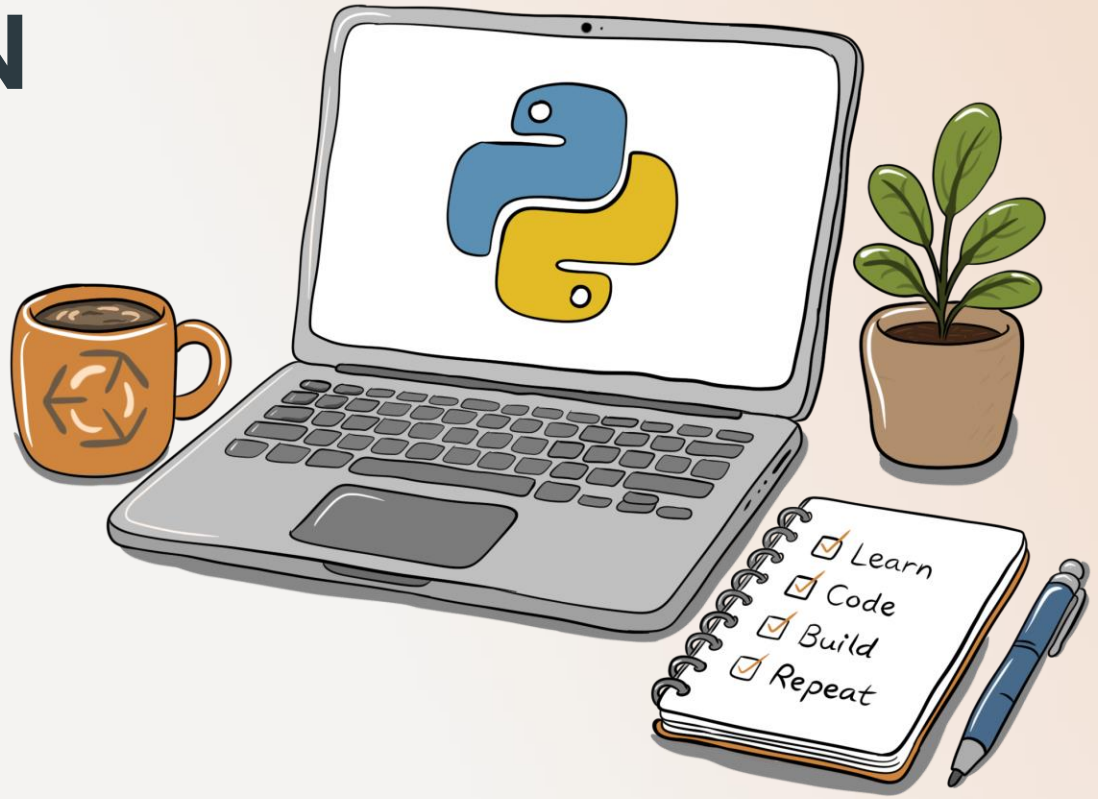
- Coding is giving a computer instructions that it runs in order
- The computer is fast, not smart: it does exactly what it is told and never guesses
- A program is like a recipe: variables are ingredients, instructions are steps, output is the dish
- A programming language is a set of keywords and rules, and each language has its own
- The CPU only runs machine code; a compiler or interpreter translates our code



Key takeaway
Be exact. The computer follows your instructions to the letter, in the order you give them.

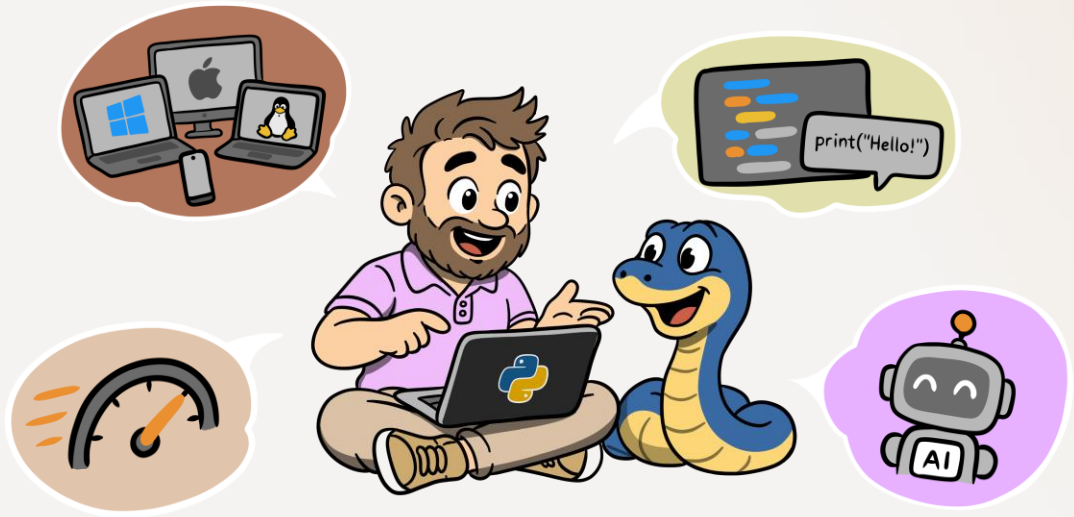
INTRODUCING PYTHON

- Why Python?
- Python setup
- Python CLI
- Setting up VSCode



WHY PYTHON?

- It runs on almost every machine: Windows, Mac, Linux, even tiny devices.
- It is easy to read and easy to write. The code looks close to plain English.
- It is one of the fastest languages for a beginner to pick up and get results with.
- It is everywhere in AI and machine learning, which is a fast-growing field worth getting into early.



LESS CODE, SAME RESULT

helloWorld.java

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World");
    }
}
```

main.py

```python
print("Hello, World")
```

Bottom line:
Same output. Python is one line. Java needs five lines of structure before it even prints. For a beginner, less to type means less to get wrong.

TestHuset

INSTALLING PYTHON ON WINDOWS

1. Go to python.org and open the Downloads page. Download the latest version for Windows.
2. Run the installer.
3. Important: tick the box "Add python.exe to PATH" at the bottom before you click Install. This lets you run Python from any terminal.
4. Click "Install Now" and wait for it to finish.
5. To confirm it worked, open a new terminal and the next slide shows what to type.

CHECK THE INSTALL (command line)

- Open the Windows terminal (search for "Command Prompt" or "PowerShell" in the Start menu).
- Type this and press Enter:

```
python --version
```

- You should see something like:

```
Python 3.13.0
```

- If you see a version number, Python is installed. If you see an error, the PATH box was probably missed during install.

THE PYTHON COMMAND LINE (REPL)

- Type python in the terminal and press Enter. The prompt changes to >>>. You are now inside Python.
- Whatever you type, Python runs immediately and shows the result. Great for trying things out



- Type exit() and press Enter to leave and go back to the normal terminal.

TestHuset

SETTING UP VISUAL STUDIO CODE

1. Download and install Visual Studio Code from code.visualstudio.com. It is free.
2. Open VS Code, go to the Extensions panel (the squares icon on the left), search for "Python" and install the official one from Microsoft.
3. Create a new file and save it with a .py ending, for example hello.py.
4. Type your code:



5. Press the Run button (the triangle, top right) and watch the output appear in the panel at the bottom.

TestHuset

RECAP: INTRODUCING PYTHON

- Python runs almost everywhere and reads close to plain English
- Less code for the same result means less to type and less to get wrong
- Install from python.org and tick “Add python.exe to PATH”
- Check the install in the terminal, then use the REPL (>>>) to try things out
- VS Code with the official Python extension is where we write our programs

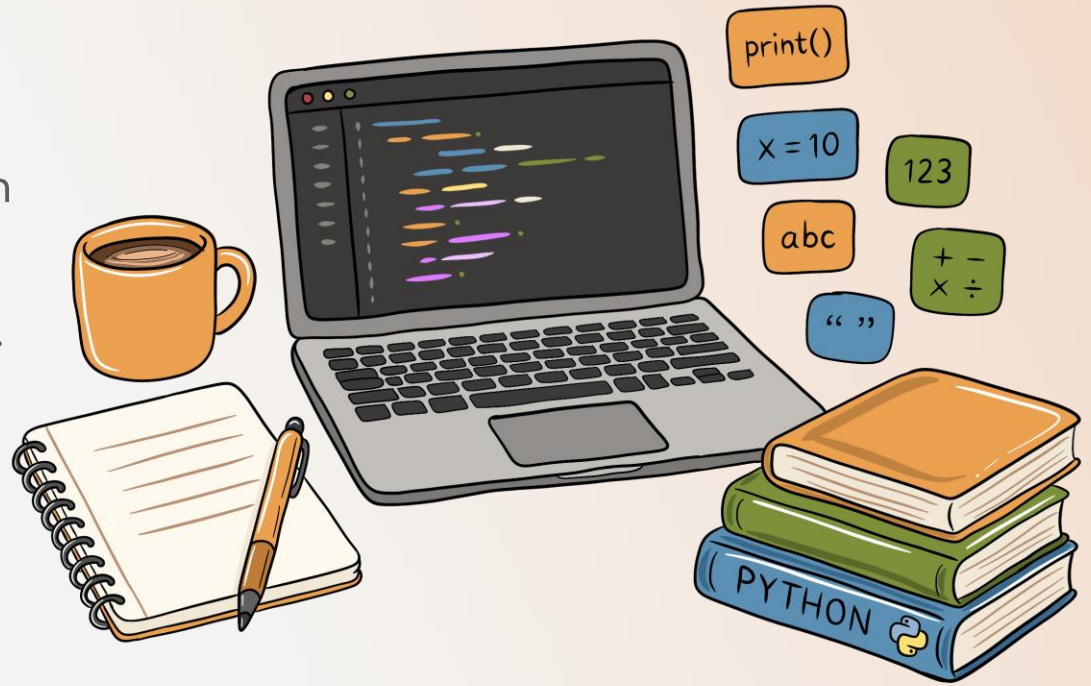


Key takeaway
Your environment is ready: Python is installed and verified, and VS Code is set up.

BASIC SYNTAX

The building blocks, hands on

- We will learn the core pieces every Python program is made of.
- After each idea, you write a tiny bit a code yourself.
- Goal: by the end of this section, you can read and write simple Python without help.



SHOWING THINGS: print()

- `print()` displays whatever you put inside the brackets
- Text goes in single or double quotes. Numbers do not need quotes.

main.py

```
print("Hello!")
print(42)
print("The answer is", 42)
```

- **Exercise #1:** Print your own name, then print your age on a second line.



COMMENTS

- A line starting with # is a comment. Python ignores it completely.
- Comments explain code to people. They never affect what the program does.

main.py

```
# This is a comment
print("This runs")   # comments can go at the end of a line too
```

- **Exercise #2:** Add a comment above your name-printing code that says what it does.



VARIABLES

- A variable is a named box that holds a value. Use = to put a value in it.
- After that, the name stands in for the value.

main.py

```
name = "Alex"
age = 32

print(name, "is", age)
```

- **Exercise #3:** Make a variable city with your, then print ‘I live in’ followed by your city.



GETTING INPUT

- `input()` pauses the program and waits for the user to type something.
- Important: `input()` always gives back text, even if they type a number.
- To do math with it, convert first: `int(...)` for whole numbers, `float(...)` for decimals.

main.py

```
name = input("What is your name? ")
print("Hello,", name)

age = int(input("Your age? "))   # convert text to a number
print("Next year you will be", age + 1)
```

- **Exercise #4:** Ask the user for two numbers, then print their sum.



ARITHMETIC

- + add, - subtract, * multiply, / divide.
- ** is power, % (modulo) gives the remainder.

main.py

```
print(7 + 3)    # 10
print(7 / 2)    # 3.5
print(7 // 2)   # 3   (whole-number division)
print(7 % 2)    # 1   (remainder)
print(2 ** 3)   # 8   (2 to the power 3)
```



- **Exercise #5**: Calculate how many full weeks are in 100 days, and how many days are left over.

WORKING WITH STRINGS

- You can join text with +, but everything must be a string.
- Easier way: an f-string. Put f before the quotes and drop variables inside { }.



main.py

```
name = "Alex"
age = 32

print("Hi " + name)                    # joining with +
print(f"Hi {name}, you are {age}")     # f-string, much cleaner
```

- **Exercise #6**: Using an f-string, print "In 5 years, {name} will be {age + 5}".

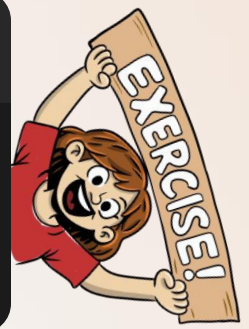
INDENTATION MATTERS

- In Python, indentation (the space at the start of a line) is part of the rules, not just for looks.
- Code that belongs together is indented the same amount. Getting it wrong is an error.



main.py

```
if age >= 18:
    print("Adult")      # indented: belongs to the if
print("Done")           # not indented: always runs
```



- **Exercise #7**: Write an if that prints "Hello" only when a variable mood equals "happy".

CAPSTONE: TIP CALCULATOR

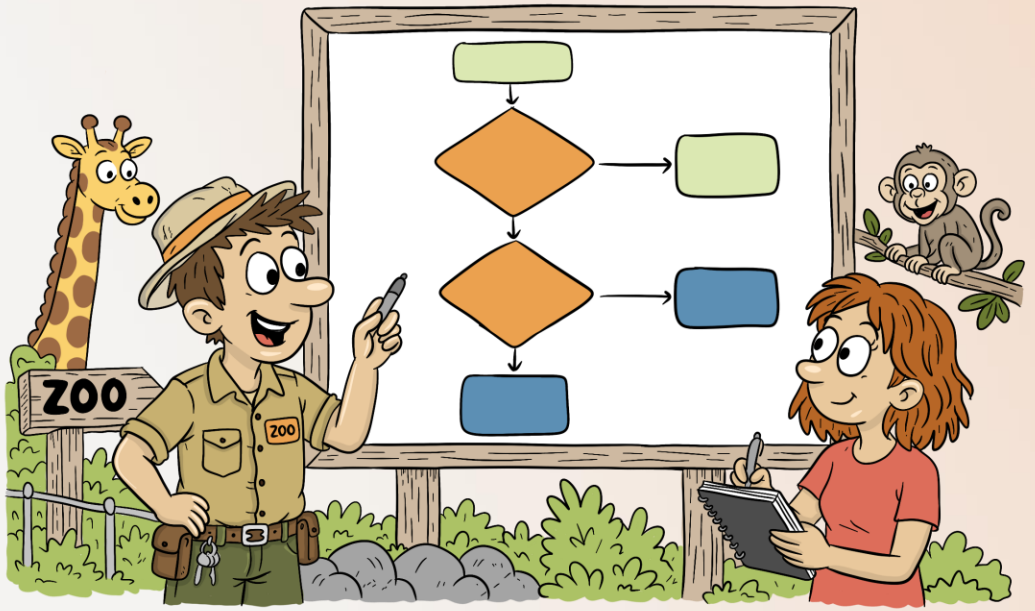
- Ask the user for the bill amount
- Ask for a tip percentage
- Calculate the tip
- Calculate the total bill
- Display the tip and total



CONDITIONALS

Teaching your program to make decisions

- Until now, every program ran every line, top to bottom. That changes today.
- We will learn how programs ask questions and choose between different paths.
- After each idea, you write a tiny bit of code yourself.
- Goal: by the end of this section, you build a complete zoo ticket system on your own.



COMPARISON OPERATORS

- A comparison takes two values and answers with True or False.
- These are the same booleans you met in the data types slide.

main.py

```
print(5 > 3)
print(5 < 3)
print(5 == 5)
print(5 != 4)
print(5 >= 5)
print(3 <= 2)
```

- **Exericse #1:** Make a variable temperature = 25. Print whether it is above 20, and whether it is exactly 30.



WATCH OUT: = is not ==

One equals stores, two equals asks

- `=` is assignment. It puts a value into a variable
- `==` is comparison. It asks, “are these the same?” and answers True or False



- If your `if` line errors or behaves strangely, check this first

CONDITIONAL CODE WITH: `if`

- An if statement runs code only when its condition is True.
- The condition is any expression that answers True or False, like a comparison.
- Note the colon at the end of the if line. It is required.

main.py

```
age = 20

if age >= 18:
    print("You are an adult")

print("This line always runs")
```



Exercise #2: Ask the user for a number. If it is greater than 100, print "That is a big number".

CODE BLOCKS

- The indented lines after the `if` form a code block. They run, or get skipped, together.
- The first line that is not indented is outside the block and always runs.
- Standard indentation is four spaces. VS Code does it for you when you type the colon and press Enter.

main.py

```
mood = "happy"

if mood == "happy":
    print("Great to hear!")        # in the block
    print("Keep smiling")          # also in the block

print("Goodbye")                   # outside, always runs
```



- **Exercise #3:** Write an `if` with a block of three print lines, and one line after it that always runs. Test it with the condition both True and False.

OTHERWISE: ELSE

- else gives the program a second path. If the condition is False, the else block runs instead.
- Exactly one of the two blocks always runs. Never both.

main.py

```
age = int(input("Your age? "))

if age >= 18:
    print("Entry allowed")
else:
    print("Sorry, adults only")
```

- **Exercise #4:** Ask for a password. If it equals "secret123", print "Welcome". Otherwise print "Access denied".



ELIF

- elif means "else, if". It lets you check several conditions in order.
- Python tests them top to bottom and runs the first block that matches, then skips the rest.
- In many languages this is written else if. In Python it is one word: elif.

main.py

```
score = int(input("Your score? "))

if score >= 90:
    print("Grade: A")
elif score >= 75:
    print("Grade: B")
elif score >= 60:
    print("Grade: C")
else:
    print("Grade: F")
```

Exercise #5: Ask for a temperature. Below 0 print "Freezing", 0 to 20 print "Cool", above 20 print "Warm".

LOGICAL OPERATORS

Combing questions: and, or, not

- and is True only when both sides are True
- or is True when at least one side is True
- not flips True to False and back



- **Exercise #6:** Ask for an age. Print "Teenager" only if age is at least 13 and under 20.

NESTED CONDITIONS

A decision inside a decision

- An `if` can live inside another if's block. The inner question is only asked when the outer one passed.
- Each level adds one more indent.
- Use nesting when the second question only makes sense after the first. If you are just combining requirements, and is usually cleaner.

main.py

```
has_ticket = True
age = 10

if has_ticket:
    if age < 12:
        print("Child entry: please bring an adult")
    else:
        print("Entry allowed")
else:
    print("Please buy a ticket first")
```

- **Exercise 7#:** Ask if the user has an umbrella (yes or no). If yes, ask if it is raining. Only if both, print "You stay dry".

CAPSTONE: The Zoo Entry System

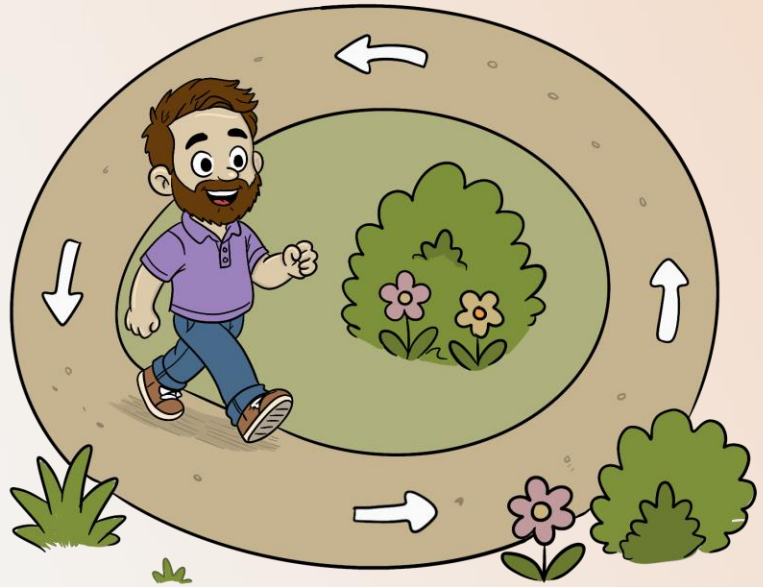
The task: Build the ticket system for a zoo. Your program should

- Ask the visitor's age
- Pick a price:
  - Under 3 is free
  - 3 – 12 pays 50 DKK
  - 13 – 64 pays 120 DKK
  - 65 and above pays 80 DKK
- Ask if they are member (Boolean). Members get 20 DKK off. But only is the ticket being not already free.
- Ask which animal they came to see. The lion exhibit is closed today: If they answer "lion", suggest the tigers instead.
- If the visitor is under 13, ask if an adult is with them. If not, politely refuse entry.
- Print a ticket summary with the final price.

LOOPS

Making your program repeat work

- So far, every line runs once. But real programs often need to repeat something many times.
- Loops let you run the same block of code repeatedly, without copying it.
- After each idea, you write a tiny bit of code yourself
- Goal: by the end of this section, you build a working number guessing game on your own.



DOING IT AGAIN: ITERATION

- Iteration means repeating a set of instructions. Each repeat is called one iteration.
- A loop keeps repeating as long as it is told to, then stops.
- Two questions every loop answers: what do I repeat, and when do I stop?

main.py

```
# Without a loop (tedious)
print("Hello")
print("Hello")
print("Hello")
```



- **Exercise #1:** Talk it through, no code yet. How many times would you write print to count to 50?

REPEAT WHILE TRUE: `while`

- A `while` loop repeats its block as long as a condition stays True.
- It checks the condition, runs the block, checks again, and so on.
- Something inside the loop must eventually make the condition False, or it never stops.

main.py

```
count = 1

while count <= 5:
    print(count)
    count = count + 1     # this line is what ends the loop

print("Done")
```



- **Exercise #2:** Use a while loop to print the numbers from 1 to 10.

TestHuset

AVOIDING INFINITE LOOPS

- An infinite loop is a loop whose condition never becomes False. The program freezes and keeps going forever.
- Most often the cause is forgetting to change the variable the condition checks.
- If your program hangs, you can stop it with Ctrl + C in the terminal.

main.py

```
count = 1

while count <= 5:
    print(count)
    # forgot count = count + 1, so count stays 1 forever
```



- **Exercise #3:** Run the broken loop above on purpose. Watch it spam, stop it with Ctrl + C, then add the missing line to fix it

HOLDING MANY VALUES: `lists`

- A list stores several values in one variable, written in square brackets.
- The values can be text, numbers, anything. They keep their order.
- We need lists because the next loop, for-in, loves to walk through them.

main.py

```
animals = ["lion", "tiger", "bear"]

print(animals)        # the whole list
print(animals[0])     # "lion"  (counting starts at 0)
```

- **Exercise #4:** Make a list of three of your favourite foods and print the first one.



TestHuset

THE for-in LOOP

- A for-in loop runs its block once for every item in a collection.
- Each time around, the loop variable holds the next item.
- No counter to manage, and no risk of an infinite loop. It stops when the items run out.

main.py

```
animals = ["lion", "tiger", "bear"]

for animal in animals:
    print("The zoo has a", animal)
```

- **Exercise #5:** Make a list of three names and use a for-in loop to print "Hello" to each one.



COUNTING WITH: range()

- `range()` produces a sequence of numbers to loop over.
- `range(5)` gives 0, 1, 2, 3, 4. It starts at 0 and stops before the number you give.
- You can also give a start and end: range(1, 6) gives 1, 2, 3, 4, 5.

main.py

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for i in range(1, 6):
    print(i)          # 1 2 3 4 5
```

- **Exercise #6:** Use range() to print the 7 times table from 7 up to 70.



THE ACCUMULATOR PATTERN

- An accumulator is a variable you update a little on each iteration to build up a result.
- Start it before the loop, change it inside the loop, use it after.
- This is how you total, count, or average a series of values.

main.py

```
total = 0                    # start before the loop

for number in [10, 20, 30]:
    total = total + number   # add each one

print(total)                 # 60
```

- **Exercise #7**: Use a loop and an accumulator to add up all the numbers from 1 to 100.



TestHuset

STEERING A LOOP: BREAK AND CONTINUE

- break stops the loop immediately and moves on, even if there is more to do.
- continue skips the rest of this iteration and jumps to the next one.
- break is the clean way to end a while True loop when something happens, like the user typing "quit".

main.py

```
while True:                          # looks infinite, but...
    command = input("Type a command (or 'quit'): ")
    if command == "quit":
        break                        # ...break gives us the exit
    print("You said:", command)
```



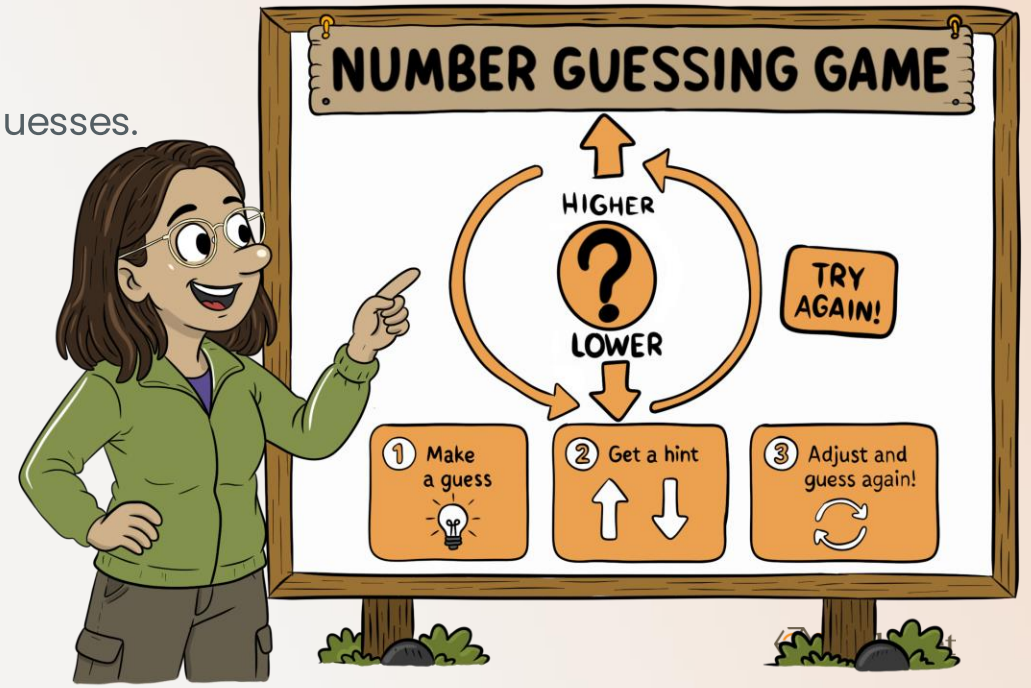
- **Exercise #8:** Loop forever asking for a word. Print each word, but stop the loop when the user types "stop".

CAPSTONE: GUESSING GAME

- **The task:** The computer picks a secret number and the player tries to guess it. Your program should:
  1. Pick a random secret number between 1 and 20.
  2. Keep asking the player to guess until they get it right.
  3. After each guess, say whether the real number is higher or lower.
  4. Count how many guesses they used.
  5. When they win, congratulate them and show the number of guesses.
- One new tool: to get a random number, add this at the top:

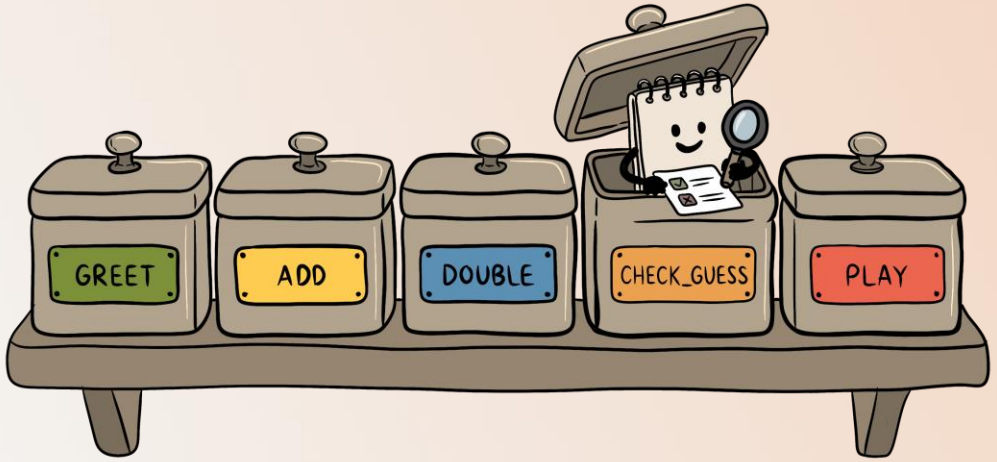
main.py

```
import random
secret = random.randint(1, 20)   # a random whole number, 1 to 20
```



FUNCTIONS

Naming and reusing blocks of code



- A function is a named block of code you can run whenever you want, as many times as you want.
- You write it once, then call it by name instead of repeating yourself.
- After each idea, you write a tiny bit of code yourself.
- Goal: by the end of this section, you take the guessing game you already built and rebuild it cleanly out of functions.

WHY FUNCTIONS?

- Without functions, code you need twice gets copied twice. Fix a bug, and you have to fix it in every copy.
- Functions let you write the logic once and reuse it everywhere by name.
- They also give messy code a clear shape: each function does one job with a name that says what it is.

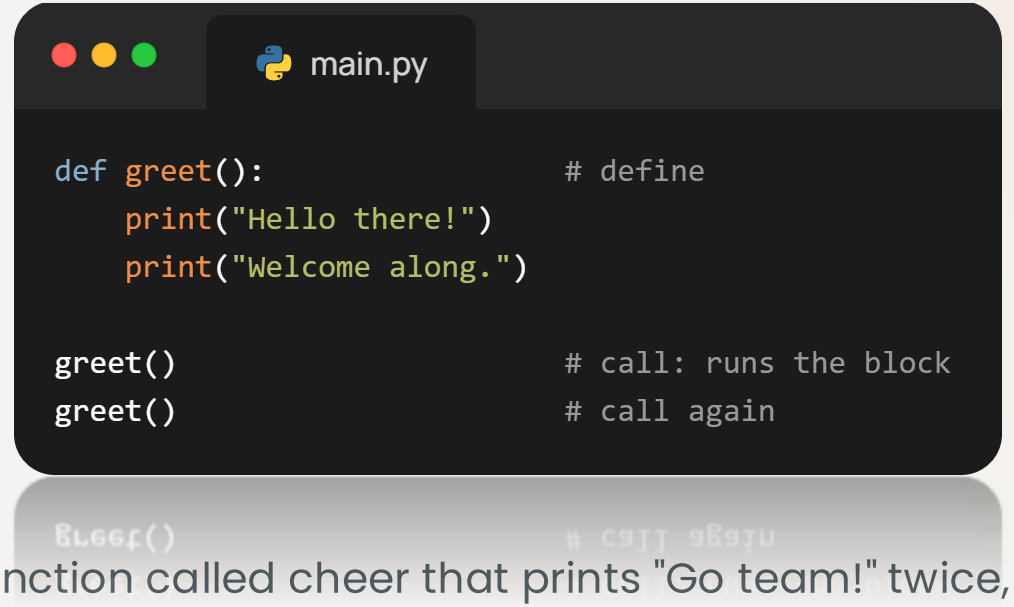
main.py

```
# Repeated work, copied three times
print("Welcome, please sign in")
print("Welcome, please sign in")
print("Welcome, please sign in")
```

- **Exercise #1:** If that welcome message had to change, how many lines would you edit?

DEFINING AND CALLING

- You define a function with def, a name, and a block of code. Defining it does not run it.
- You run it by calling it: the name followed by brackets.
- Define is writing the recipe. Calling is actually cooking it.

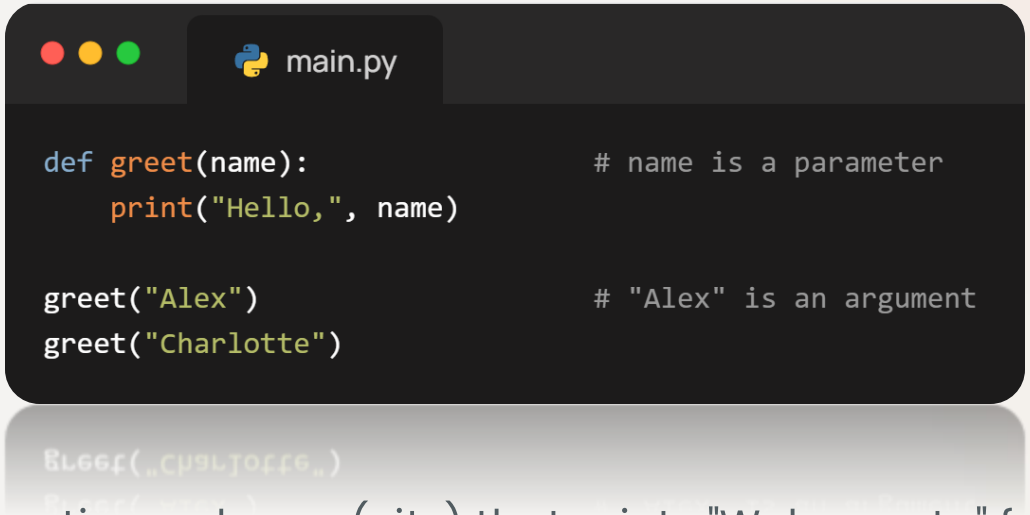


- **Exercise #2:** Write a function called cheer that prints "Go team!" twice, then call it three times.

TestHuset

PARAMETERS AND ARGUMENTS

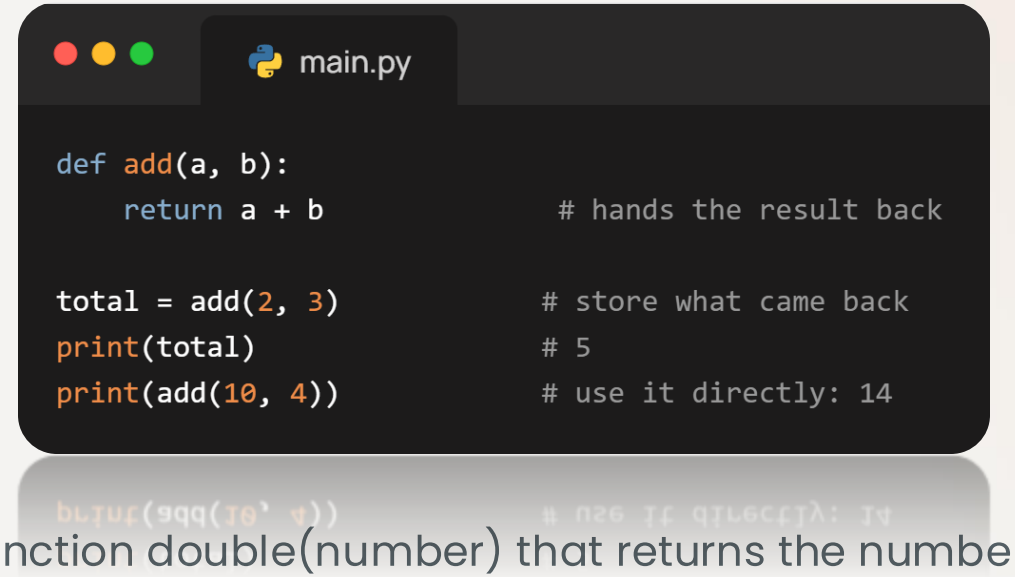
- A parameter is a placeholder you put in the brackets when you define a function.
- An argument is the real value you give when you call it.
- This lets one function do its job on different values.



- **Exercise #3:** Write a function welcome(city) that prints "Welcome to" followed by the city. Call it with two different cities.

RETURN VS PRINT

- `print` shows something on the screen. That is all it does.
- `return` hands a value back to whoever called the function, so it can be stored and reused.
- A printed value is gone after it appears. A returned value you can keep working with.

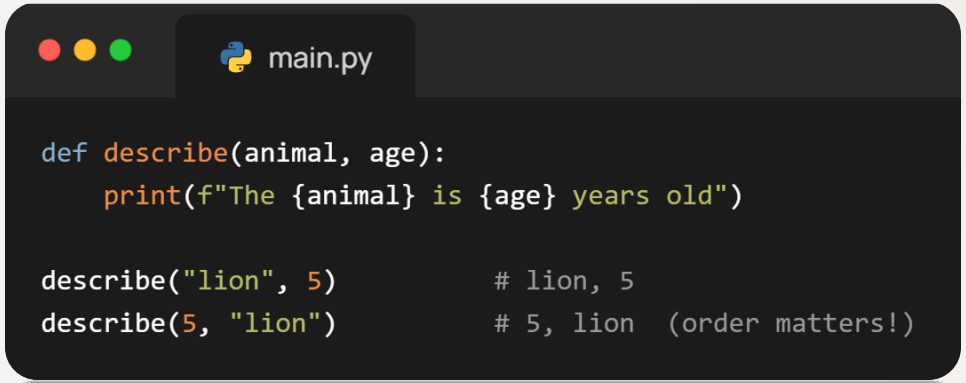


- **Exercise #4:** Write a function double(number) that returns the number times two. Store the result of double(8) in a variable and print it.

TestHuset

MULTIPLE PARAMETERS

- A function can take several parameters, separated by commas.
- The arguments you pass line up with the parameters in order.
- Get the order wrong and the function still runs, just with the values in the wrong places.

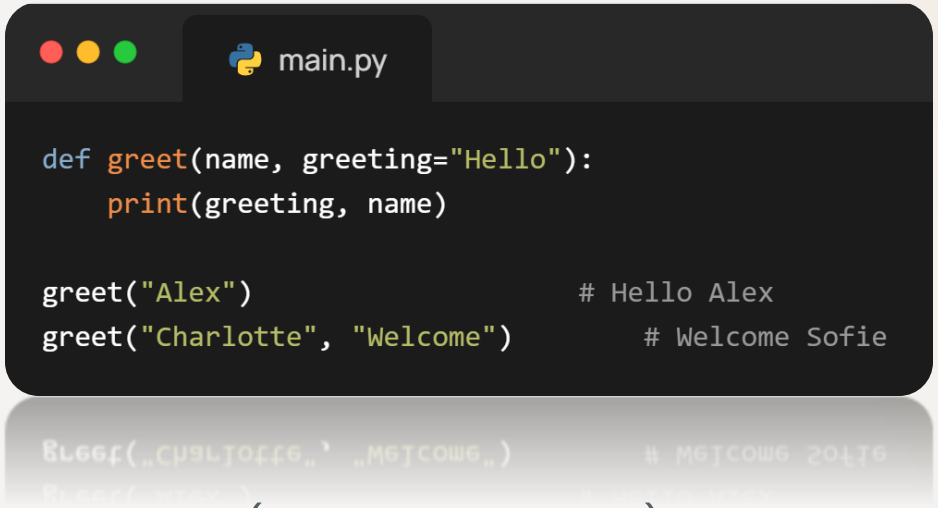


- **Exercise #5:** Write a function rectangle_area(width, height) that returns width times height. Print the area of a 4 by 6 rectangle.

TestHuset

DEFAULT PARAMETERS

- You can give a parameter a default value in the definition.
- If the caller leaves it out, the default is used. If they provide one, it wins.
- Handy when a value is usually the same but you want to allow exceptions.



- **Exercise #6:** Write a function power(base, exponent=2) that returns base to the exponent. Call it once with just a base, and once with both.

CAPSTONE: REFACTOR GUESSING GAME

- **The task:** Take the guessing game from the last section and reshape it into functions. Aim for these:
  1. pick_secret() returns a random number from 1 to 20.
  2. get_guess() asks the player and returns their number.
  3. check_guess(guess, secret) prints "Higher" or "Lower", and returns True if the guess was correct.
  4. play_game() ties it all together: get the secret, loop until the player wins, count the attempts.
- Then call play_game() to start.